



기존에 작성하던 투두메이트 프로그램에 무한 루프 메뉴를 적용하고, 기능을 최소 3개의 독립된 함수로 분리해줘. 이때 매개변수, return문, global 키워드를 각각 1회 이상 무조건 포함해야 하고 메인 코드는 간결해야 해.

기존에 작성하던 투두메이트(Todo Mate) 프로그램의 핵심 기능을 깔끔하게 정리하고, 요구사항(무한 루프 메뉴, 3개 이상의 독립된 함수, 매개변수 / return문 / global 키워드 필수 사용)을 모두 반영한 파이썬 코드 구조입니다.

메인 코드는 최대한 간결하게 유지하고, 각 함수가 독립된 역할을 수행하도록 구성했습니다.

투두메이트 리팩토링 코드

python



```
# 할 일 목록을 저장할 전역 변수
todo_list = []

# 함수 1: 할 일 추가 (global 키워드 활용)
def add_todo():
    """사용자로부터 할 일을 입력받아 전역 변수 리스트에 추가합니다."""
    global todo_list # 전역 변수 todo_list를 변경하기 위해 global 키워드 사용
    task = input("새로운 할 일을 입력하세요: ").strip()
    if task:
        todo_list.append({"task": task, "completed": False})
        print(f" '{task}' 목록에 추가되었습니다.\n")
    else:
        print(" 내용을 입력하지 않았습니다.\n")

# 함수 2: 할 일 완료 표시 및 상태 변경 (매개변수 활용)
def complete_todo(index):
    """전달받은 인덱스의 할 일 상태를 완료(True)로 변경합니다."""
    # 인덱스 범위가 유효한지 확인
    if 0 <= index < len(todo_list):
        todo_list[index]["completed"] = True
        print(f" '{todo_list[index]['task']}' 일을 완료 처리했습니다! 🎉\n")
    else:
        print(" 올바른지 않은 번호입니다.\n")

# 함수 3: 할 일 목록 문자열 생성 (return문 활용)
def get_todo_status_str():
    """현재 할 일 목록의 상태를 보기 좋은 문자열로 만들어 반환합니다."""
    if not todo_list:
        return "할 일 목록이 비어 있습니다. 새로운 할 일을 추가해 보세요!\n"

    status_str = ""
    for i, item in enumerate(todo_list):
        status_str += f"{i + 1}. [{item['completed']}] {item['task']}\n"

    result_str = status_str
    return result_str
```

+ Gemini에게 물어보세요





요구사항 반영 포인트 설명

- 무한 루프 메뉴: `main()` 함수 내부에서 `while True:` 를 통해 사용자가 4번을 눌러 `break` 하기 전까지 메뉴가 계속 반복됩니다.
- 3개 이상의 독립된 함수: `add_todo()`, `complete_todo()`, `get_todo_status_str()` 3개로 명확하게 역할을 나누었습니다.
- 핵심 키워드 만족:
 - `global`: `add_todo()` 내부에서 함수 바깥에 있는 전역 변수 리스트(`todo_list`)에 직접 요소를 수정-추가하기 위해 `global todo_list` 를 사용했습니다.
 - 매개변수 (Parameter): `complete_todo(index)` 에서 어떤 번호의 할 일을 완료할지 외부(메인 코드)로부터 `index` 값을 넘겨받습니다.
 - `return` 문: `get_todo_status_str()` 에서 화면에 뿌려줄 문자열을 직접 출력하지 않고, `return result_str` 을 통해 메인 코드로 결과값을 돌려줍니다.
- 간결한 메인 코드: 흐름을 제어하는 `main()` 함수는 사용자의 입력(`choice`)에 따라 매칭되는 함수만 쏙쏙 호출하므로 전체 구조가 한눈에 들어옵니다.



`complete_todo` 함수에서 사용자가 번호를 잘못 입력하거나 리스트가 빌 때의 예외 처리를 `return`문 조건에 맞춰서 보완해줘.

`complete_todo` 함수가 단순히 결과를 직접 출력(`print`)하는 대신, 성공 여부나 예외 상황에 대한 결과 메시지를 `return` 하여 메인 코드에서 출력하도록 보완했습니다.

또한, 리스트가 비어있을 때 처리를 추가하고 조건문 구조를 다듬어 예외 처리를 한층 더 강화했습니다.



수정된 `complete_todo` 함수 코드

python

```
def complete_todo(index):
```

```
    """
```

```
    전달받은 인덱스의 할 일 상태를 완료로 변경합니다.
```

```
    작업 결과에 대한 메시지를 리턴합니다.
```

```
    """
```

```
    # 예외 처리: 1. 할 일 목록이 비어있는 경우
```



Gemini에게 물어보세요



```
if index < 0 or index >= len(todo_list):
```

```
def complete_todo(index):
```

```
    """
```

```
    전달받은 인덱스의 할 일 상태를 완료로 변경합니다.
```

```
    작업 결과에 대한 메시지를 리턴합니다.
```

```
    """
```

```
    # 예외 처리 1: 할 일 목록이 비어있는 경우
```

```
    if not todo_list:
```

```
        return f"❌ 현재 등록된 할 일이 없어 완료할 수 없습니다."
```

```
    # 예외 처리 2: 인덱스 범위를 벗어난 경우 (잘못된 번호 입력)
```

```
    if index < 0 or index >= len(todo_list):
```

```
        return f"❌ 올바른지 않은 번호입니다. (선택 가능한 번호: 1 ~ {len(todo_list)})"
```

```
    # 예외 처리 3: 이미 완료된 일을 다시 완료하려고 하는 경우 (선택적 예외 처리)
```

```
    if todo_list[index]["completed"]:
```

```
        return f"⚠️ '{todo_list[index]['task']}' 목록은 이미 완료된 상태입니다."
```

```
    # 모든 예외를 통과하면 정상 처리 후 완료 메시지 리턴
```

```
    todo_list[index]["completed"] = True
```

```
    return f"👏 '{todo_list[index]['task']}' 일을 완료 처리했습니다!"
```

🔄 메인 코드(main) 연동 부분 수정하기

`complete_todo` 함수가 이제 메시지를 `return` 하므로, 메인 코드에서는 함수를 호출한 결과를 `print()` 로 감싸서 출력해주어야 합니다.

```
python
```



```
elif choice == "3":
```

```
    # 목록을 먼저 보여준 후 번호 선택 유도
```

```
    print(get_todo_status_str())
```

```
    # 리스트가 비어있어도 complete_todo 내부에서 안전하게 처리되므로 바로 입력받아도 됨
```

```
    try:
```

```
        num = int(input("완료할 할 일의 번호를 입력하세요: "))
```

```
        # 함수가 리턴한 메시지를 받아와서 화면에 출력!
```

```
        result_message = complete_todo(num - 1)
```

```
        print(f"\n{result_message}\n")
```

```
    except ValueError:
```

```
        print(f"❌ 올바른 숫자를 입력해 주세요.\n")
```

💡 무엇이 보완되었나요?

- 역할이 분리 (수정 전후 비교)

+ Gemini에게 물어보세요



입니다.

-

메인 코드가 이 값을 받아 화면에 보여주거나 다른 용도로 활용하기 훨씬 좋아졌습니다.



Gemini Flash



```
todo_list[index]["completed"] = True
return f"🎉 '{todo_list[index]['task']}' 일을 완료 처리했습니다!"
```

메인 코드(main) 연동 부분 수정하기

complete_todo 함수가 이제 메시지를 return 하므로, 메인 코드에서는 함수를 호출한 결과를 print() 로 감싸서 출력해주어야 합니다.

python



```
elif choice == "3":
    # 목록을 먼저 보여준 후 번호 선택 유도
    print(get_todo_status_str())

    # 리스트가 비어있어도 complete_todo 내부에서 안전하게 처리되므로 바로 입력받아도 됨
    try:
        num = int(input("완료할 할 일의 번호를 입력하세요: "))
        # 함수가 리턴한 메시지를 받아와서 화면에 출력!
        result_message = complete_todo(num - 1)
        print(f"\n{result_message}\n")

    except ValueError:
        print("❌ 올바른 숫자를 입력해 주세요.\n")
```

💡 무엇이 보완되었나요?

- 역할의 분리 (수정 전후 비교):
 - 기존에는 함수 내부에서 직접 print() 를 해버려서 메인 코드가 함수의 흐름을 제어하기 어려웠습니다.
 - 수정 후에는 함수가 처리에 성공했든 실패했든 그 결과를 return 문 조건에 맞춰 텍스트로 반환하므로, 메인 코드가 이 값을 받아 화면에 보여주거나 다른 용도로 활용하기 훨씬 좋아졌습니다.
- 비어있는 리스트 방어: 목록에 아무것도 없을 때 무작정 번호를 입력받아 터지는 상황을 if not todo_list: 조건으로 가장 먼저 걸러내어 안전합니다.
- 친절한 피드백: 유저가 인덱스를 잘못 입력했을 때, 단순히 에러 메시지만 띄우는 것이 아니라 (선택 가능한 번호: 1 ~ N) 처럼 현재 선택 가능한 범위를 동적으로 계산해서 알려주도록 디테일을 더했습니다.



Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. [개인정보보호](#) 및 [Gemini](#)

+ Gemini에게 물어보세요





이렇게 오류가 났는데 이유가 뭐지

보내주신 터미널 화면(마지막 이미지)을 보니 `UnboundLocalError: cannot access local variable 'status' where it is not associated with a value` 에러가 명확하게 찍혀 있네요!

이 에러가 발생한 원인은 우리가 아까 Troubleshooting 회고를 작성하면서 예측했던 변수 이름 오타 때문이 맞습니다. 파이썬이 왜 이 단계에서 멈췄는지 직관적으로 이유를 설명해 드릴게요.

🔍 에러가 터진 진짜 이유

에러가 발생한 위치는 `main.py` 파일의 35번째 줄(`print(f"{i + 1}. {status} ...")`)입니다.

화면에 1. 할 일 입력 을 통해 일을 등록한 뒤 2. 투두 조회 메뉴를 누르셨을 텐데, 이때 컴퓨터는 `show_todos` 함수 내부의 조건문을 돌리게 됩니다.

python

```
# [예상되는 현재 코드 상태]
if item["done"] == True:
    status = "[완료]"
else:
    statua = "[미완료]" # 🐞 [원인] 여기에 'status'가 아니라 'statua'로 오타가 나 있습니다

print(f"{i + 1}. {status} {item['task']}") # 🐞 [35번째 줄] 파이썬이 status를 출력하
```

새로 등록한 할 일은 기본적으로 미완료 상태이기 때문에 컴퓨터는 `else:` 블록으로 들어갑니다. 그런데 `else:` 안에서 실제로 `** statua = "[미완료]"`라고 철자 오타를 내는 바람에, 컴퓨터는 `status` 라는 변수를 만들지 못하고 `statua` 라는 엉뚱한 이름으로 데이터를 저장해 버린 것이죠.

그 직후 바로 아래 줄에서 `status` 를 출력하라고 하니, 파이썬 입장에서는 `***할당된(생성된) 적이 없는 status` 라는 변수를 갑자기 가져오라고 하니 접근할 수 없다!***라며 에러를 뿜고 프로그램을 강제 종료(Traceback)시킨 것입니다.



해결 방법

1 주 + Gemini에게 물어보세요



2 main.py

3 else: 바로 아래줄에 적히 ** statua **를 원래 의도했던 이름인 ** status **로 철자를 올바르게 고쳐줍니